

25

TF-IDF

Turning words into numbers the classic way

Where Natural Language Processing begins

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A way to turn text into meaningful numbers
Full name	Term Frequency – Inverse Document Frequency
Core idea	Important words are frequent here but rare elsewhere
Best for	Search, classification, simple text tasks
Your project	Twitter Sentiment (TF-IDF beat the deep models!)

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	Why Turn Words into Numbers?	Models can't read text
2	Bag of Words First	The simplest approach
3	The Problem with Counting	Common words drown out the rest
4	TF — Term Frequency	How often a word appears
5	IDF — Inverse Document Frequency	How rare a word is
6	Putting Them Together	The TF-IDF score
7	The Document-Term Matrix	Text becomes a table of numbers
8	TF-IDF in Code	A real scikit-learn example
9	Strengths and Limits	When to use it
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. Why Turn Words into Numbers?

This is the start of Natural Language Processing, or NLP — teaching computers to work with human language. There's one immediate problem: machine learning models only understand numbers, not words. Before any model can classify a tweet or detect spam, the text has to become numbers.

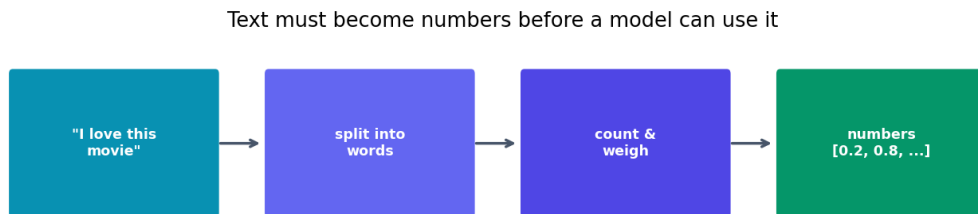


Fig 1 — Text must be converted into numbers before any model can use it. TF-IDF is one classic way to do this.

TF-IDF is one of the oldest and most reliable ways to do this conversion. Despite being simple and decades old, it still works remarkably well — in your own Twitter Sentiment project, TF-IDF with Logistic Regression actually beat the fancy deep learning models. This topic explains why it's so effective.

■ *Your Twitter Sentiment project is the perfect example: the classical TF-IDF approach outperformed RNN/LSTM/GRU. That's not a fluke — for many text tasks, TF-IDF is genuinely strong. A great lesson that simpler is sometimes better.*

2. Bag of Words First

The simplest way to turn text into numbers is Bag of Words. You make a list of every word in your vocabulary, then for each document you count how many times each word appears. The order is ignored — you just throw all the words in a 'bag' and count them.

```
# Two short documents:
doc1 = 'the movie was great great'
doc2 = 'the movie was boring'

# Bag of Words counts:
#           the  movie  was  great  boring
# doc1:      1     1    1     2     0
# doc2:      1     1    1     0     1
```

Now each document is a row of numbers a model can use. This works, but it has a weakness that TF-IDF was invented to fix.

3. The Problem with Plain Counting

With plain counting, the most common words get the biggest numbers. But the most common words — 'the', 'is', 'a', 'was' — are usually the least meaningful. They appear in almost every document and tell you nothing about what makes a document special.

Meanwhile, a rare but meaningful word like 'brilliant' or 'terrible' might appear just once, getting a tiny count, even though it's the most important word for understanding the sentiment. Plain counting gets it backwards: it rewards boring common words and ignores rare meaningful ones.

■ *The goal of TF-IDF is to fix this: boost words that are meaningful (frequent in THIS document but rare overall) and shrink words that are everywhere (like 'the'). It finds the words that actually matter.*

4. TF — Term Frequency

The first half is Term Frequency (TF). It simply measures how often a word appears in one document. A word used many times in a document is probably important to that document. This is the same as plain counting, usually divided by the document length so longer documents don't get unfair advantage.

$$\text{TF} = (\text{times the word appears}) / (\text{total words in the document})$$

Example: if 'movie' appears 3 times in a 100-word review, its TF is $3/100 = 0.03$. Simple. But TF alone still has the common-word problem — 'the' would have a high TF too. That's where the second half comes in.

5. IDF — Inverse Document Frequency

The second half is Inverse Document Frequency (IDF). It measures how rare a word is across ALL your documents. The idea: a word that appears in every document (like 'the') is not useful for telling documents apart, so it gets a low score. A word that appears in only a few documents is special, so it gets a high score.

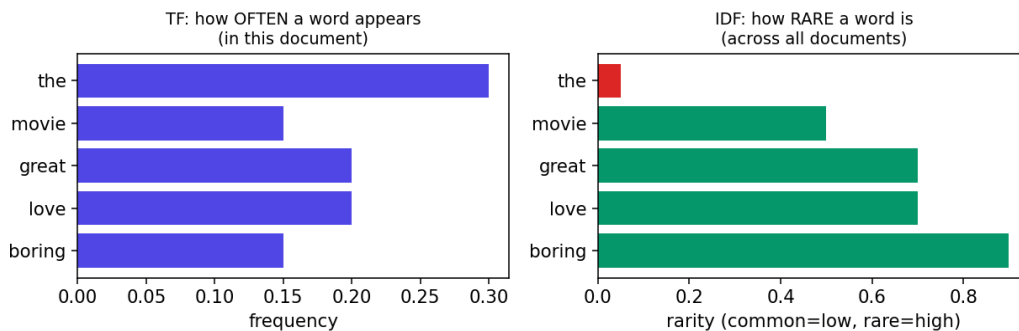


Fig 2 — TF measures how often a word appears here. IDF measures how rare it is overall. Common words (red) get low IDF.

$$\text{IDF} = \log(\text{total documents} / \text{documents containing the word})$$

So 'the', which appears in every document, gets an IDF near zero — crushing its importance. A rare word like 'masterpiece' that appears in few documents gets a high IDF — boosting its importance. IDF is the part that throws out the noise of common words.

6. Putting Them Together — TF-IDF

The magic happens when you multiply the two together. A word's final TF-IDF score is high only when it's frequent in this document AND rare across all documents — exactly the words that make a document distinctive.



A word matters if it is frequent here BUT rare elsewhere

Fig 3 — $\text{TF-IDF} = \text{TF} \times \text{IDF}$. A word scores high only if it's frequent here but rare elsewhere.

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

Word type	TF	IDF	TF-IDF result
-----------	----	-----	---------------

Common word ('the')	high	near 0	LOW (crushed)
Rare unique word	high	high	HIGH (important!)
Word not in this doc	0	-	0

7. The Document-Term Matrix

When you apply TF-IDF to a whole collection of documents, you get a big table called the document-term matrix. Each row is a document, each column is a word, and each cell is that word's TF-IDF score in that document.

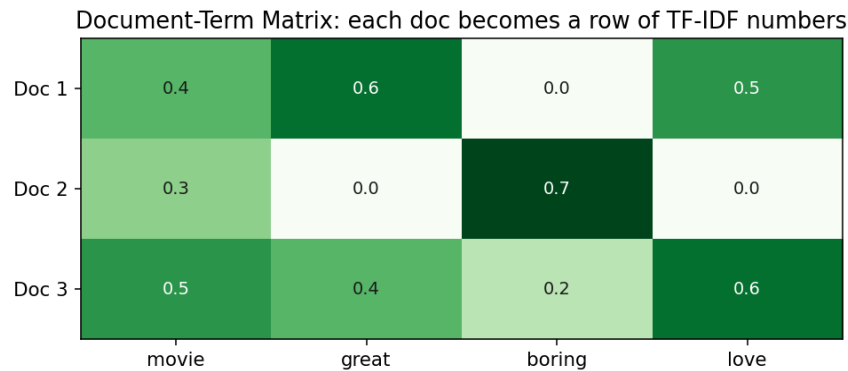


Fig 4 — Each document becomes a row of TF-IDF numbers — ready to feed into a model.

Now every document is a row of numbers — exactly what a model like Logistic Regression or XGBoost needs. This matrix is the bridge from raw text to a trained classifier. It's often 'sparse' (mostly zeros), because any single document only uses a small slice of the full vocabulary.

8. TF-IDF in Code

scikit-learn makes TF-IDF a one-liner with `TfidfVectorizer`. Here's the full flow from text to a trained sentiment classifier — essentially your Twitter project:

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

texts = ['I love this', 'terrible movie', 'great film', ...]
labels = [1, 0, 1, ...] # 1 = positive, 0 = negative

# 1. Turn text into TF-IDF numbers
vectorizer = TfidfVectorizer(
    max_features=5000, # keep the top 5000 words
    stop_words='english', # drop 'the', 'is', etc.
    ngram_range=(1, 2)) # single words AND word pairs
X = vectorizer.fit_transform(texts)

# 2. Train a simple, strong classifier
X_tr, X_te, y_tr, y_te = train_test_split(X, labels, test_size=0.2)
model = LogisticRegression()
model.fit(X_tr, y_tr)
print('Accuracy:', model.score(X_te, y_te))

# 3. Predict on new text (use transform, NOT fit_transform!)
new = vectorizer.transform(['what a wonderful film'])
print(model.predict(new))

```

■ *Key detail: use `fit_transform` on training text, but only `transform` on new text. `fit_transform` learns the vocabulary; using it again on test data would re-learn and break things. This is a very common bug.*

9. Strengths and Limits

TF-IDF is simple, fast, and surprisingly strong — but it has clear limits worth knowing.

Strengths	Limits
Simple and very fast	Ignores word order
Works with little data	'not good' looks like 'good not'
Easy to understand / explain	Doesn't understand meaning
Strong baseline for text tasks	Can't tell synonyms apart
No GPU or training needed	Big vocabulary = big sparse matrix

Because it ignores word order and meaning, TF-IDF can't tell that 'good' and 'great' are similar, or that 'not good' is negative. Modern approaches like sentence transformers (a few topics ahead) fix this by capturing meaning. But for many tasks — especially with limited data — TF-IDF remains the smart first choice. Always try it as a baseline before reaching for something heavier.

■ *Using `ngram_range=(1,2)` helps with the word-order weakness a little: it captures word PAIRS like 'not good' as a single feature. This small trick often boosts accuracy on sentiment tasks.*

10. Common Mistakes

Mistake	Why it's a problem	Fix
<code>fit_transform</code> on test data	Re-learns vocab, leaks info	transform only on test/new text
Keeping all words	Huge, noisy matrix	Use <code>max_features</code> , <code>stop_words</code>
Ignoring word pairs	Misses 'not good'	Use <code>ngram_range=(1,2)</code>
Expecting it to understand	It only counts, not comprehends	Use embeddings for meaning
Not lowercasing	'Movie' != 'movie'	<code>TfidfVectorizer</code> lowercases by default
Skipping it for deep learning	May lose a strong baseline	Always try TF-IDF first

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
TF	how often a word appears in a document
IDF	how rare a word is across all documents
TF-IDF	$TF \times IDF$ = a word's importance
Bag of Words	plain word counts (no weighting)
Document-term matrix	table of TF-IDF numbers

Import	<code>from sklearn.feature_extraction.text import TfidfVectorizer</code>
Create	<code>TfidfVectorizer(max_features=5000, stop_words='english')</code>
Learn + transform (train)	<code>vectorizer.fit_transform(texts)</code>
Transform (new text)	<code>vectorizer.transform(new_texts)</code>
Word pairs	<code>ngram_range=(1, 2)</code>
See the words	<code>vectorizer.get_feature_names_out()</code>
Pairs well with	LogisticRegression, LinearSVC

The one idea to remember: TF-IDF turns text into numbers by scoring each word on two things — how often it appears here (TF) and how rare it is overall (IDF). Words that are frequent here but rare elsewhere score highest, capturing what makes a document distinctive.

Next Topic: Hugging Face — the library and community that gives you instant access to thousands of powerful pre-trained Transformer models with just a few lines of code.